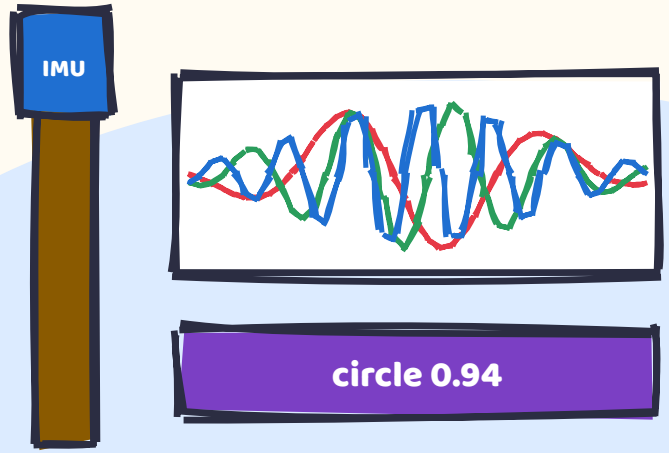
Tech Leaders · Class 19

TinyML Gesture Recogniser on ESP32

A neural network smaller than a photo, running on a $5 chip.

TinyML · Data · Train · Deploy



Did you know?
Billions of microcontrollers ship every year. TinyML puts small AI models on them: in earbuds, watches and sensors.

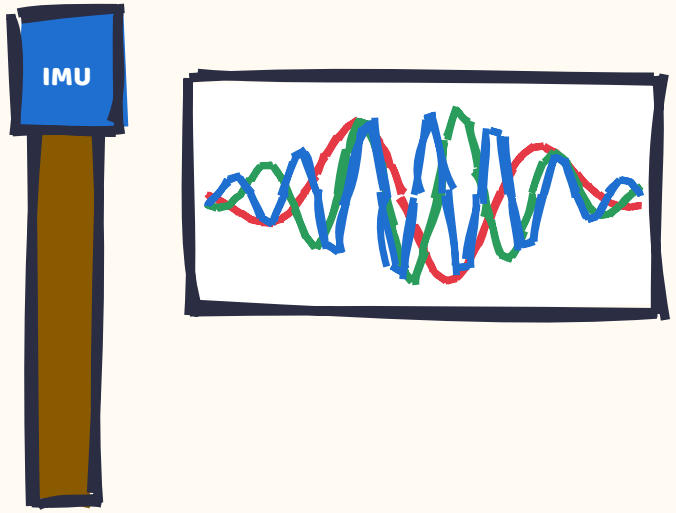
LEARN

Let's think...

Does a smartwatch need the cloud?

No internet needed

Your watch counts steps and spots a fall even in airplane mode. A wake-word chip hears 'Hey...' without sending audio anywhere. How can a tiny chip with 520 KB of RAM run AI?



It can't!

It cheats?

Tiny models!

Today: you train a **gesture-recognising neural network** on motion data and run it **on an ESP32**: a magic wand with no laptop.

LEARN

Learning objectives

By the end of this class I can...

1	**Why TinyML:** explain why running AI **on a microcontroller** helps **latency, privacy and power**	**Proof:** I can give 3 reasons a smartwatch runs AI on-device
2	**Collect data:** record **labelled motion data** at a fixed sample rate with an MPU6050	**Proof:** my Edge Impulse project has 3+ minutes of data per gesture
3	**Train + evaluate:** train a model with **spectral features** and judge it with a **confusion matrix**	**Proof:** my test accuracy is above 85% and I know which gestures get confused
4	**Deploy + act:** deploy to the **ESP32** and trigger actions above a **confidence threshold**	**Proof:** my wand lights an LED for 'circle' with no laptop

LEARN

Objective 1 · Why TinyML · video

Video time!



What is TinyML?

Arm · about 6 minutes · youtu.be/Y1qwcPqxkp0

Watch for...

1. What makes TinyML different from cloud AI?
2. Why does **power** matter so much?
3. Name two TinyML examples from the video.

Scan to watch

After the video: Which device in your home could become smarter with TinyML?

LEARN

Objective 1 · Why TinyML

Why run AI on the device?

The idea

Cloud AI: send data to a big server, wait for the answer. **TinyML**: run a small model on the microcontroller itself.

Latency: answer in milliseconds, no network trip.
Privacy: raw data never leaves the device.
Power: radios use far more energy than maths; a coin cell can last months.
Offline: works in a forest, a factory, or airplane mode.

Trade-off: models must be **tiny** (KBs, not GBs).

Example: a reflex vs asking your parents



ms latency

data stays local

µW to mW power

Touch a hot pan and your spinal cord pulls your hand away before your brain even knows. Local, instant decisions. TinyML is a reflex for gadgets.

LEARN

Objective 2 · Collect data

Motion data: samples and windows

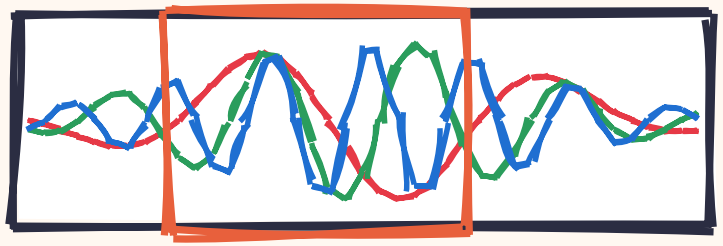
The idea

The MPU6050 gives acceleration on **x, y, z**. We sample at a fixed rate, e.g. **100 Hz** (every 10 ms).

A gesture lasts about 1 to 2 s, so we cut the stream into **windows** of e.g. **2000 ms** = 200 samples × 3 axes = **600 numbers** per example.

Each window gets a **label**: idle, circle, wave, punch. Variety matters: different people, speeds, sizes. Always include **idle**!

Example: cutting a film into clips



one 2 s window → 'circle'

A sports video is cut into 2-second clips, each labelled 'goal', 'pass', 'nothing'. The model learns from labelled clips, not the whole match.

LEARN

Objective 3 · Train + evaluate

Features, a small network, and honest testing

The idea

Raw numbers are noisy. Edge Impulse's **Spectral Analysis** block turns each window into ~30 to 40 **features**: how strong each axis is, and at which **frequencies** it moves (a wave shakes at ~2 Hz, a circle is smoother).

A small **neural network** (e.g. 20 → 10 neurons) learns to map features → label.

Keep ~20% as a **test set** the model never sees. The **confusion matrix** shows which gestures get mixed up.

Example: recognising songs by rhythm



You can tell a qawwali from a pop song by its rhythm and beat, without hearing every word. Spectral features are the 'rhythm' of a movement.

Objective 4 · Deploy + act

Deploy and act with confidence

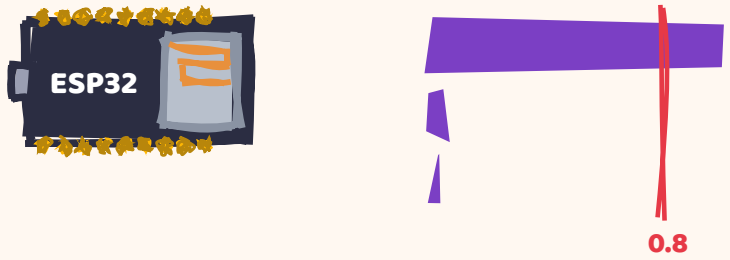
The idea

Edge Impulse exports your model as an **Arduino library** (C++, quantised to **int8**: 4× smaller than float). On the ESP32 it runs in a few **milliseconds**.

Every window the model outputs probabilities, e.g. circle 0.94, wave 0.04, idle 0.02.

Only act if the top score > a **threshold** (e.g. 0.8), and maybe require it **twice in a row**. This stops false triggers from random movement.

Example: a referee's decision



A good referee only blows the whistle when sure. Unsure? Play on. A threshold stops your gadget 'blowing the whistle' at every twitch.

LEARN

Real world

TinyML is everywhere



Fall detection
Watches detect a fall and call for help



Wake words
'Hey ...' detectors run on a tiny always-on chip



Machine health
Vibration sensors predict when a motor will break



Wildlife
Sound sensors detect chainsaws or gunshots in forests



What gesture or motion would be useful to detect in a factory, hospital or farm in Pakistan?

Small is powerful
Many TinyML models are smaller than a single phone photo, yet run for months on a battery.

LEARN

Quick quiz

TinyML check

Question	Answer
1. Three reasons to run AI on-device?	
2. 100 Hz for 2 s, 3 axes: how many numbers?	
3. Why hold back a test set?	
4. Why include an 'idle' class?	
5. Model says circle 0.6. Act if threshold is 0.8?	

Answers

Cover! 1: latency, privacy, power (offline) · 2: 600 · 3: honest accuracy on unseen data · 4: so normal movement isn't forced into a gesture · 5: no

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human classifier
2. Our TinyML kit
3. Stream data to Edge Impulse
4. Design, train and test the impulse
5. Run the model on the ESP32
6. Model report
7. ML engineer challenges

MAKE

Class game

Human classifier

How to play

1. Volunteers each invent a gesture for 'circle', 'wave' and 'punch' and perform it 5 times.
2. The class (the 'model') watches only a **shadow** on the wall (a torch behind them): that's limited sensor data!
3. Teacher then shows NEW people's gestures. The class votes with a confidence: hands up 1 to 5 fingers.
4. Which gestures got confused? What extra 'training' helped?

Then discuss

Why did new people trick the class more than the original volunteers?



Big idea

Variety in training data = a model that works for everyone.

MAKE

Build it

Our TinyML kit

ESP32 dev board
Runs the model

MPU6050
SDA 21, SCL 22, 3V3, GND

Edge Impulse (free)
studio.edgeimpulse.com
(13+ with permission)

Edge Impulse CLI
edge-impulse-data-forwarder (Node.js)

Wand / glove
Tape the board + IMU to a stick

LED / buzzer
To show the result

Mount the IMU firmly: if it wobbles on the stick, your data is noise.

Accounts
Edge Impulse needs an account. Use the class account set up by the teacher if you're under the site's minimum age.

MAKE

Objective 2 · Collect data · Build step 1 of 3

Stream data to Edge Impulse

```
#include <Adafruit_MPU6050.h>
#include <Wire.h>
Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!mpu.begin()) { Serial.println("MPU not found"); while (1); }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
}

void loop() {
  static unsigned long next = 0;
  if (millis() >= next) {          // exactly every 10 ms = 100 Hz
    next = millis() + 10;
    sensors_event_t a, g, t;
    mpu.getEvent(&a, &g, &t);
    Serial.print(a.acceleration.x); Serial.print(',');
    Serial.print(a.acceleration.y); Serial.print(',');
    Serial.println(a.acceleration.z);
  }
}
// Laptop: edge-impulse-data-forwarder  → name axes ax, ay, az
```

Forwarder

The CLI detects 100 Hz and 3 axes, then links your board to your project.

Record

Data acquisition → label 'circle', length 10 s, do circles. Repeat for each class.

Amount

Aim for 3+ minutes per class, from at least 3 people.

MAKE

Objective 3 · Build step 2 of 3

Design, train and test the impulse

a Create impulse: **window 2000 ms**, increase 200 ms → **Spectral Analysis** → **Classification** (Keras).

b Generate features. Look at the **feature explorer**: are your classes separate clusters?

c Train (e.g. 30 to 50 epochs). Note validation accuracy.

d Run **Model testing** on the held-back data. Read the **confusion matrix**.

e Weak class? Collect more varied data for it, not just more epochs.

idle	48	1	1	0
circle	0	44	4	2
wave	1	5	43	1
punch	0	1	0	49

Check: Test accuracy > 85% and you can name the most confused pair.

MAKE

Objective 4 · Deploy + act · Build step 3 of 3

Run the model on the ESP32

```ino
#include <gesture_wand_inferencing.h>   // from Deployment → Arduino library
#include <Adafruit_MPU6050.h>
#include <Wire.h>
Adafruit_MPU6050 mpu;
float buf[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE];
const int LED = 2;

void setup() {
  Serial.begin(115200); Wire.begin(21, 22); mpu.begin(); pinMode(LED, OUTPUT);
}

void loop() {
  for (size_t i = 0; i < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE; i += 3) {   // fill one window
    sensors_event_t a, g, t; mpu.getEvent(&a, &g, &t);
    buf[i] = a.acceleration.x; buf[i + 1] = a.acceleration.y; buf[i + 2] = a.acceleration.z;
    delay((int)EI_CLASSIFIER_INTERVAL_MS);
  }
  signal_t signal;
  numpy::signal_from_buffer(buf, EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, &signal);
  ei_impulse_result_t res;
  if (run_classifier(&signal, &res, false) != EI_IMPULSE_OK) return;

  int best = 0;
  for (int i = 1; i < EI_CLASSIFIER_LABEL_COUNT; i++)
    if (res.classification[i].value > res.classification[best].value) best = i;
  const char *label = res.classification[best].label;
  float conf = res.classification[best].value;
  Serial.printf("%s %.2f  (%d ms)\n", label, conf, res.timing.classification);

  digitalWrite(LED, strcmp(label, "circle") == 0 && conf > 0.8);
}
```

Library

Arduino IDE → Sketch → Include Library → Add .ZIP (the Edge Impulse export).

Timing

res.timing shows how many ms the model took. Usually just a few!

Threshold

Only light the LED when confident: fewer false triggers.

MAKE

Test like an engineer

Model report

Measure	Result	Notes
Samples per class / people		
Validation accuracy		%
Test accuracy		%
Most confused pair		
On-device inference time		ms

Debug checklist

Forwarder sees wrong frequency? Use the millis() timing, not delay(10) after printing.
Great accuracy but fails live? Your test data came from the same session: record new data on a different day.
Out of memory? Use the int8 (quantised) model.

Overfitting: perfect on training data, poor on new people. More varied data fixes it better than more epochs.

MAKE

Level up!

ML engineer challenges

Choose your challenge



Spell casting: map 3 gestures to 3 actions: LED colour, buzzer tune, servo move.



Smart home: send the detected gesture over ESP-NOW to turn on a lamp.



Fall detector: collect 'fall' (onto a cushion!) vs 'sit down'. Discuss false alarms.



Audio: try a keyword spotter with a microphone instead of the IMU.

Think like an engineer

What's worse for a fall detector: a false alarm or a missed fall? How would that change your threshold?

Boss level

Add an 'anomaly detection' block so the wand ignores movements it has never seen.

Objective check

Can I really do it? Prove it.

- **01** Give three reasons to run this model on the ESP32 instead of in the cloud. ○ ○ ○
- **02** Show your dataset: how many samples, how many people, which classes? ○ ○ ○
- **03** Show your confusion matrix. Which pair is most confused, and why? ○ ○ ○
- **04** Demonstrate the wand working with no laptop attached. ○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Wizard duel: each team performs 5 gestures called by the teacher. Points for every correct detection.

Show what you know

What did we learn?

- [x] TinyML = small models on microcontrollers: fast, private, low power
- [x] Sample at a fixed rate; cut into labelled windows
- [x] Spectral features + a small neural network
- [x] Test on unseen data; read the confusion matrix
- [x] Quantise, deploy, and act above a confidence threshold

Exit ticket: why can a model score 99% in training but fail for a new person?

At home
List 3 gadgets at home that might run TinyML (earbuds, watch, phone…).



Next class:
Processing: Flocking Simulation (Boids)!